МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: «Інтелектуальні методи діагностики за медичними зображеннями»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Завдання 23: Gradient Clipping

Мета: Запобігти exploding gradients.

Хід виконання:

1. Налаштуйте clipnorm або clipvalue в оптимізаторі
2. Моніторьте величину градієнтів
3. Порівняйте стабільність навчання
4. Візуалізуйте loss curves

Очікуваний результат: Стабільніше навчання.

Теоретичні відомості

Gradient Clipping є методом стабілізації процесу навчання нейронних мереж, який використовується для запобігання проблемі exploding gradients. Під час навчання глибоких нейронних мереж градієнти можуть набувати дуже великих значень, що призводить до різких змін ваг моделі та порушення процесу оптимізації. У таких випадках модель може втрачати здатність до навчання або видавати некоректні результати.

Суть gradient clipping полягає у обмеженні величини градієнтів під час оновлення ваг. Якщо значення градієнтів перевищує заданий поріг, воно масштабується або обрізається до допустимого рівня. Це дозволяє уникнути надто великих кроків під час градієнтного спуску та забезпечує більш контрольований процес навчання.

Існують два основних підходи до gradient clipping. Перший підхід полягає у використанні clipnorm, при якому обмежується норма вектора градієнтів. Другий підхід використовує clipvalue, де обмежується кожне окреме значення градієнта. У даній роботі застосовується clipnorm, що дозволяє зберегти напрямок градієнта, але обмежити його величину.

Стабільність навчання означає плавну зміну функції втрат та відсутність різких коливань під час оптимізації. Якщо навчання є стабільним, значення loss поступово зменшується, а модель покращує свої результати без різких стрибків. Gradient clipping допомагає досягти такої поведінки, особливо у випадках складних моделей або нестабільних даних.

Хід роботи

1. Встановлення бібліотек та підготовка середовища

Було встановлено необхідні бібліотеки для роботи з нейронними мережами, обробки даних та візуалізації результатів. До них входять TensorFlow, Keras, NumPy, Pandas, Matplotlib та інші. Також було перевірено наявність графічного процесора для прискорення навчання моделі.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import os
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, optimizers, callbacks

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))
```

```
TensorFlow version: 2.19.0
GPU available: []
```

2. Генерація синтетичного датасету

Було створено синтетичний датасет медичних зображень вузлів легень. Зображення поділено на два класи, benign та malignant. Генерація виконувалася програмно, що дозволило створити навчальну, валідаційну та тестову вибірки у співвідношенні 70, 15 та 15 відсотків відповідно.

```
Dataset created in: lung_nodules
Train: 700
Val: 150
Test: 150
```

3. Завантаження та підготовка даних

Зображення було зчитано з директорій, змінено їх розмір та нормалізовано до діапазону від 0 до 1. Також було додано додатковий канал для відповідності формату входу нейронної мережі.

```python
data = load_dataset(data_dir, image_size=(64, 64))
```

```
train: (700, 64, 64, 1), labels: (700,)
val: (150, 64, 64, 1), labels: (150,)
test: (150, 64, 64, 1), labels: (150,)
```

4. Побудова архітектури нейронної мережі

Було створено згорткову нейронну мережу, яка складається з кількох згорткових шарів, шарів підвибірки та повнозв'язних шарів. Для запобігання перенавчанню використовувався Dropout.

```python
def build_simple_cnn(input_shape=(64, 64, 1), num_classes=2):
    model = models.Sequential([
        layers.Input(shape=input_shape),

        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation='softmax')
    ])
    return model
```

5. Реалізація Gradient Clipping

Було реалізовано gradient clipping через параметр clipnorm в оптимізаторі Adam. Це дозволило обмежити величину градієнтів під час навчання та запобігти їх сильному зростанню.

```python
def make_compiled_model(use_clipping=False, clipnorm=1.0, learning_rate=0.001):
    base = build_simple_cnn(input_shape=(64, 64, 1), num_classes=2)
    model = GradientMonitoringModel(base)

    if use_clipping:
        optimizer = optimizers.Adam(learning_rate=learning_rate, clipnorm=clipnorm)
    else:
        optimizer = optimizers.Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

6. Моніторинг величини градієнтів

Було реалізовано власний метод навчання, у якому обчислюється норма градієнтів на кожній ітерації. Це дозволило відстежувати поведінку градієнтів під час навчання та оцінити стабільність процесу оптимізації.

```python
class GradientMonitoringModel(tf.keras.Model):
    def __init__(self, base_model):
        super().__init__()
        self.base_model = base_model
        self.grad_norm_tracker = tf.keras.metrics.Mean(name="grad_norm")

    def call(self, inputs, training=False):
        return self.base_model(inputs, training=training)

    @property
    def metrics(self):
        return super().metrics + [self.grad_norm_tracker]

    def train_step(self, data):
        x, y = data

        with tf.GradientTape() as tape:
            y_pred = self(x, training=True)
            loss = self.compiled_loss(y, y_pred, regularization_losses=self.losses)

        grads = tape.gradient(loss, self.trainable_variables)
        valid_grads = [g for g in grads if g is not None]
        grad_norm = tf.linalg.global_norm(valid_grads)

        self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
        self.compiled_metrics.update_state(y, y_pred)
        self.grad_norm_tracker.update_state(grad_norm)

        results = {m.name: m.result() for m in self.metrics}
        results["loss"] = loss
        return results

    def test_step(self, data):
        x, y = data
        y_pred = self(x, training=False)
        loss = self.compiled_loss(y, y_pred, regularization_losses=self.losses)
        self.compiled_metrics.update_state(y, y_pred)

        results = {m.name: m.result() for m in self.metrics}
        results["loss"] = loss
        return results
```

7. Навчання моделей

Було виконано навчання двох моделей нейронної мережі. Перша модель навчалась без використання gradient clipping, тоді як друга модель навчалась із застосуванням обмеження градієнтів через параметр clipnorm. Для обох моделей використовувались однакові параметри навчання, зокрема кількість епох, розмір батчу та функції оптимізації.

У процесі навчання було використано механізми EarlyStopping та ReduceLROnPlateau, що дозволяють зупиняти навчання при відсутності покращення та автоматично зменшувати швидкість навчання. Це сприяє більш ефективній оптимізації та запобігає перенавчанню моделі.

```
Training baseline model without clipping...
Epoch 1/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 26s 1s/step - accuracy: 1.0000 - grad_norm: 0.5260 - loss: 0.1421 - val_grad_norm: 0.5000 - val_loss: 0.0014 -
Epoch 2/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 20s 913ms/step - accuracy: 0.9600 - grad_norm: 0.5406 - loss: 0.1433 - val_grad_norm: 0.5000 - val_loss: 0.117
Epoch 3/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 963ms/step - accuracy: 0.9533 - grad_norm: 0.5390 - loss: 0.0764 - val_grad_norm: 0.5000 - val_loss: 0.015
Epoch 4/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 20s 917ms/step - accuracy: 1.0000 - grad_norm: 0.5217 - loss: 0.1796 - val_grad_norm: 0.5000 - val_loss: 0.006
Epoch 5/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 923ms/step - accuracy: 0.9856 - grad_norm: 0.5131 - loss: 0.0594
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 969ms/step - accuracy: 1.0000 - grad_norm: 0.5232 - loss: 0.0530 - val_grad_norm: 0.5000 - val_loss: 0.002
Epoch 6/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 883ms/step - accuracy: 1.0000 - grad_norm: 0.5156 - loss: 0.0869 - val_grad_norm: 0.5000 - val_loss: 1.779
Epoch 7/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 959ms/step - accuracy: 1.0000 - grad_norm: 0.5093 - loss: 0.0142 - val_grad_norm: 0.5000 - val_loss: 0.001
Epoch 8/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 39s 883ms/step - accuracy: 1.0000 - grad_norm: 0.5009 - loss: 1.6956e-04 - val_grad_norm: 0.5000 - val_loss: 6
Epoch 9/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 964ms/step - accuracy: 1.0000 - grad_norm: 0.5103 - loss: 0.0013 - val_grad_norm: 0.5000 - val_loss: 0.002
Epoch 10/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 828ms/step - accuracy: 0.9971 - grad_norm: 0.5046 - loss: 0.0233
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 875ms/step - accuracy: 1.0000 - grad_norm: 0.5076 - loss: 8.7266e-05 - val_grad_norm: 0.5000 - val_loss: 6
Epoch 11/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 981ms/step - accuracy: 1.0000 - grad_norm: 0.5055 - loss: 0.0781 - val_grad_norm: 0.5000 - val_loss: 8.985
Epoch 12/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 882ms/step - accuracy: 1.0000 - grad_norm: 0.5018 - loss: 0.0013 - val_grad_norm: 0.5000 - val_loss: 0.001
Epoch 13/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 968ms/step - accuracy: 1.0000 - grad_norm: 0.5071 - loss: 4.8270e-04 - val_grad_norm: 0.5000 - val_loss: 7
Epoch 14/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 839ms/step - accuracy: 0.9997 - grad_norm: 0.4965 - loss: 0.0065
Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 886ms/step - accuracy: 1.0000 - grad_norm: 0.4989 - loss: 0.0096 - val_grad_norm: 0.5000 - val_loss: 6.225
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 6.
```

Ta

```
Training clipped model with clipnorm=1.0...
Epoch 1/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 24s 913ms/step - accuracy: 0.7733 - grad_norm: 0.5179 - loss: 0.4792 - val_grad_norm: 0.5000 - val_loss: 0.1763
Epoch 2/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 974ms/step - accuracy: 0.9733 - grad_norm: 0.5701 - loss: 0.1827 - val_grad_norm: 0.5000 - val_loss: 0.0076
Epoch 3/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 20s 899ms/step - accuracy: 0.9600 - grad_norm: 0.5574 - loss: 0.1006 - val_grad_norm: 0.5000 - val_loss: 0.0943
Epoch 4/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 897ms/step - accuracy: 0.9618 - grad_norm: 0.5553 - loss: 0.1281
Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 950ms/step - accuracy: 0.9800 - grad_norm: 0.5748 - loss: 0.1420 - val_grad_norm: 0.5000 - val_loss: 0.0014
Epoch 5/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 41s 968ms/step - accuracy: 1.0000 - grad_norm: 0.5694 - loss: 0.0161 - val_grad_norm: 0.5000 - val_loss: 0.0309
Epoch 6/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 20s 899ms/step - accuracy: 1.0000 - grad_norm: 0.5444 - loss: 0.0535 - val_grad_norm: 0.5000 - val_loss: 0.0098
Epoch 7/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 952ms/step - accuracy: 1.0000 - grad_norm: 0.5149 - loss: 0.0092 - val_grad_norm: 0.5000 - val_loss: 9.4727e
Epoch 8/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 843ms/step - accuracy: 0.9894 - grad_norm: 0.5175 - loss: 0.0448
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
22/22 ━━━━━━━━━━━━━━━━━━━━ 39s 889ms/step - accuracy: 1.0000 - grad_norm: 0.5297 - loss: 0.0852 - val_grad_norm: 0.5000 - val_loss: 1.3592e
Epoch 9/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 946ms/step - accuracy: 0.9733 - grad_norm: 0.5127 - loss: 0.0082 - val_grad_norm: 0.5000 - val_loss: 7.7338e
Epoch 10/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 41s 964ms/step - accuracy: 1.0000 - grad_norm: 0.5122 - loss: 0.0118 - val_grad_norm: 0.5000 - val_loss: 3.6389e
Epoch 11/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 20s 906ms/step - accuracy: 1.0000 - grad_norm: 0.5146 - loss: 0.0402 - val_grad_norm: 0.5000 - val_loss: 4.4891e
Epoch 12/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 900ms/step - accuracy: 0.9870 - grad_norm: 0.5157 - loss: 0.0270
Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 947ms/step - accuracy: 1.0000 - grad_norm: 0.5088 - loss: 0.0250 - val_grad_norm: 0.5000 - val_loss: 7.11
Epoch 13/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 889ms/step - accuracy: 1.0000 - grad_norm: 0.5058 - loss: 0.0061 - val_grad_norm: 0.5000 - val_loss: 8.16
Epoch 14/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 930ms/step - accuracy: 1.0000 - grad_norm: 0.5107 - loss: 0.0032 - val_grad_norm: 0.5000 - val_loss: 0.06
Epoch 15/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 40s 894ms/step - accuracy: 1.0000 - grad_norm: 0.5027 - loss: 0.0015 - val_grad_norm: 0.5000 - val_loss: 2.28
Epoch 16/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 908ms/step - accuracy: 0.9938 - grad_norm: 0.5045 - loss: 0.0161
Epoch 16: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 970ms/step - accuracy: 1.0000 - grad_norm: 0.5038 - loss: 7.6437e-04 - val_grad_norm: 0.5000 - val_loss:
Epoch 17/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 19s 885ms/step - accuracy: 1.0000 - grad_norm: 0.5038 - loss: 0.0420 - val_grad_norm: 0.5000 - val_loss: 1.58
Epoch 18/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 22s 959ms/step - accuracy: 1.0000 - grad_norm: 0.5037 - loss: 0.0100 - val_grad_norm: 0.5000 - val_loss: 2.21
Epoch 19/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 40s 903ms/step - accuracy: 1.0000 - grad_norm: 0.5012 - loss: 0.0472 - val_grad_norm: 0.5000 - val_loss: 1.76
Epoch 20/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 874ms/step - accuracy: 1.0000 - grad_norm: 0.4978 - loss: 0.0062
Epoch 20: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
22/22 ━━━━━━━━━━━━━━━━━━━━ 21s 950ms/step - accuracy: 1.0000 - grad_norm: 0.4976 - loss: 0.0023 - val_grad_norm: 0.5000 - val_loss: 1.06
Epoch 21/25
22/22 ━━━━━━━━━━━━━━━━━━━━ 41s 975ms/step - accuracy: 1.0000 - grad_norm: 0.5037 - loss: 0.0152 - val_grad_norm: 0.5000 - val_loss: 1.33
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 13.
```

У ході навчання обидві моделі продемонстрували швидку збіжність. Значення accuracy вже на ранніх етапах наближались до одиниці, що свідчить про легкість задачі класифікації. Значення функції втрат зменшувались до дуже малих значень, а learning rate автоматично зменшувався у моменти стабілізації. Норма градієнтів залишалась стабільною на рівні близько 0.5, що свідчить про відсутність проблеми exploding gradients.

8. Оцінка результатів

Було виконано оцінку якості моделей за допомогою основних метрик класифікації, таких як accuracy, precision, recall та F1 score. Дані метрики дозволяють комплексно оцінити ефективність моделі при розв’язанні задачі бінарної класифікації.

```
clipped_results = evaluate_model_full(clipped_model, data
```

```
...
Baseline model
--------------------------------------------------
Accuracy : 1.0000
Precision: 1.0000
Recall   : 1.0000
F1-score : 1.0000

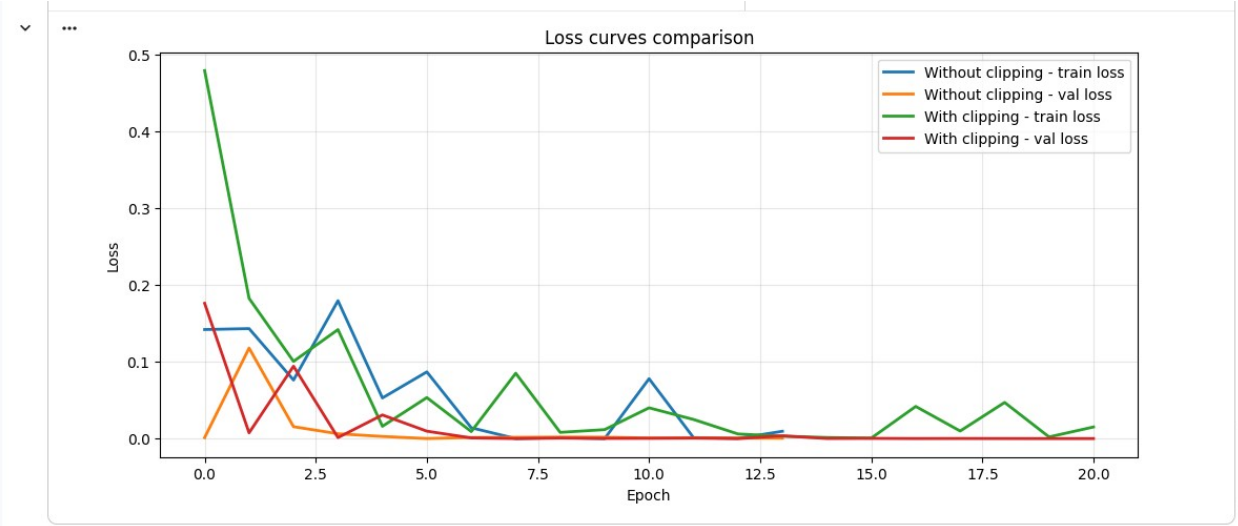
Clipped model
--------------------------------------------------
Accuracy : 1.0000
Precision: 1.0000
Recall   : 1.0000
F1-score : 1.0000
```

Отримані результати показали, що обидві моделі досягли ідеальних значень метрик, зокрема accuracy, precision, recall та F1 score дорівнюють 1.0000. Це означає, що всі тестові зразки були класифіковані правильно.

Такі результати пояснюються використанням синтетичного датасету, у якому класи мають чітко виражені відмінності. У зв’язку з цим задача класифікації є спрощеною, і модель легко знаходить закономірності у даних. У результаті навіть базова модель без gradient clipping демонструє максимально можливу точність.

9. Візуалізація loss curves

Було побудовано графіки зміни функції втрат для обох моделей, що дозволяє проаналізувати процес навчання та оцінити його стабільність.

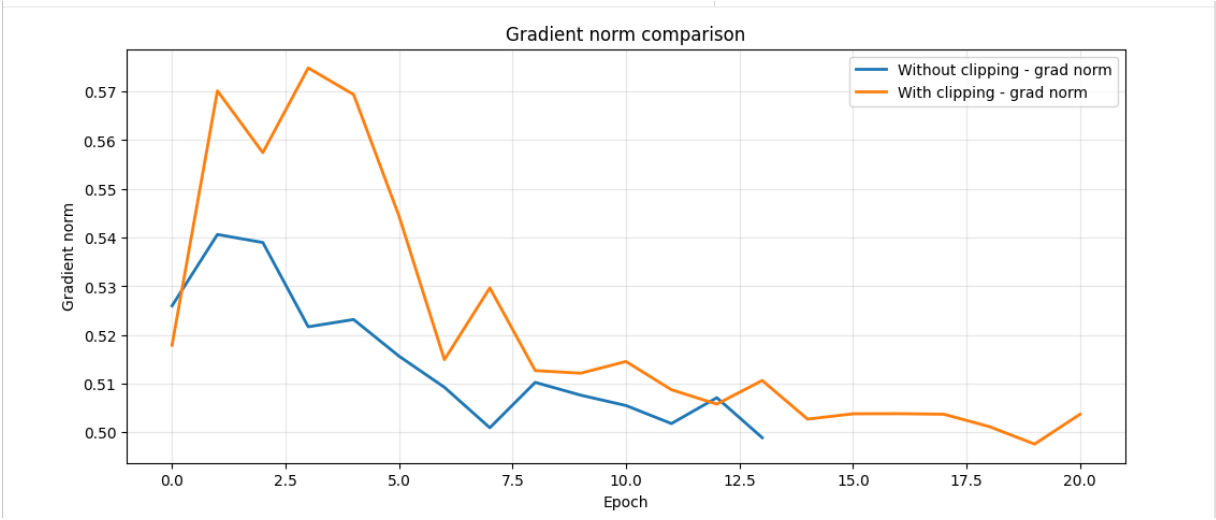


Аналіз графіків показує, що значення функції втрат для обох моделей швидко зменшується та стабілізується на дуже малих значеннях. Відсутні різкі коливання або стрибки, що свідчить про стабільний процес навчання.

Порівняння моделей показує, що різниця між використанням gradient clipping та його відсутністю є незначною. Це пояснюється тим, що в даному випадку задача є простою, і проблема exploding gradients не виникає навіть без застосування clipping.

10. Візуалізація величини градієнтів

Було побудовано графік зміни норми градієнтів під час навчання для обох моделей. Це дозволяє безпосередньо оцінити стабільність процесу оптимізації.

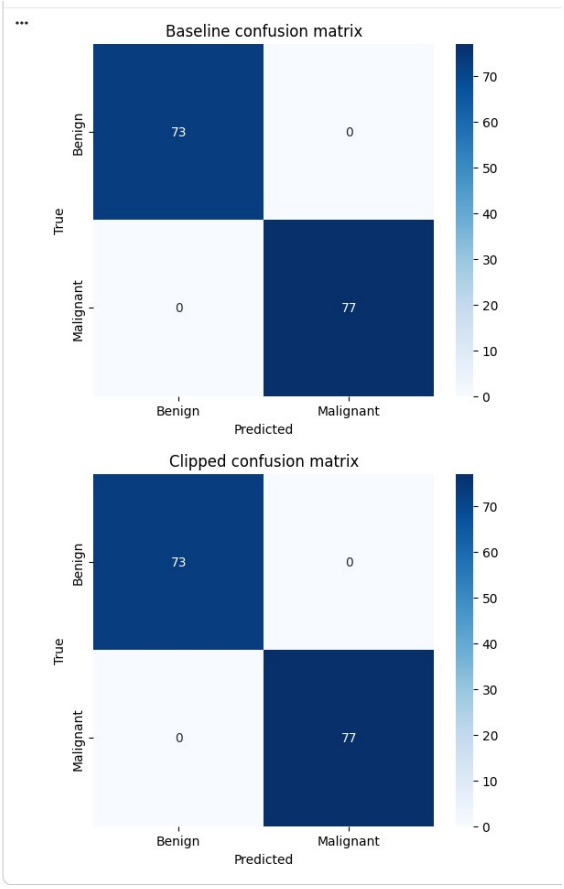


З графіка видно, що значення норми градієнтів для обох моделей знаходяться в стабільному діапазоні приблизно від 0.5 до 0.57. Відсутні різкі зростання або аномальні значення, що свідчить про відсутність проблеми exploding gradients.

Модель із gradient clipping демонструє трохи більш контрольовану поведінку градієнтів, однак у даному випадку різниця є незначною через простоту задачі.

11. Побудова матриці плутанини

Було побудовано матрицю плутанини для обох моделей з метою детального аналізу результатів класифікації.



Матриця плутанини показує, що обидві моделі правильно класифікували всі зразки. Значення на головній діагоналі дорівнюють 73 та 77 відповідно, що відповідає кількості об’єктів кожного класу. При цьому відсутні помилки класифікації, що підтверджує ідеальні значення метрик.

Це ще раз підтверджує, що модель повністю навчилась розрізняти класи на даному датасеті, що пояснюється його синтетичною природою та чіткими відмінностями між класами.

Висновок

У ході виконання лабораторної роботи було реалізовано згорткову нейронну мережу для класифікації медичних зображень вузлів легень на два класи: benign та malignant. Було виконано повний цикл роботи з даними: згенеровано синтетичний датасет, здійснено його розподіл на навчальну, валідаційну та тестову вибірки, проведено нормалізацію та підготовку даних. Побудована CNN-модель відповідала архітектурі, наведеній у методичних вказівках, та була успішно навчена із застосуванням оптимізатора Adam і відповідних метрик оцінювання.

Було досліджено вплив gradient clipping на стабільність навчання моделі. Для цього реалізовано два підходи: навчання без обмеження градієнтів та навчання з використанням параметра clipnorm в оптимізаторі. Додатково було організовано моніторинг величини градієнтів під час навчання шляхом обчислення їх глобальної норми на кожній ітерації. Це дозволило проаналізувати поведінку градієнтів і оцінити потенційну наявність проблеми exploding gradients.

За результатами експериментів встановлено, що обидві моделі продемонстрували дуже високі значення метрик якості, зокрема accuracy, precision, recall та F1-score, що пояснюється використанням синтетичного датасету з чітко вираженими відмінностями між класами. Візуалізація loss curves показала швидку збіжність моделей, а графік норми градієнтів підтвердив відсутність значних коливань під час навчання. У зв’язку з цим вплив gradient clipping у даному випадку виявився незначним, оскільки навчання і без нього було стабільним.

Посилання на Goole Colab:
https://colab.research.google.com/drive/1sRC6a5oIa_aSufFOjExexxwdJ4tiYJJG?usp=sharing